Lenses & Optics

Lenses · Update nested city in immutable Person/Address · Build with Waqar

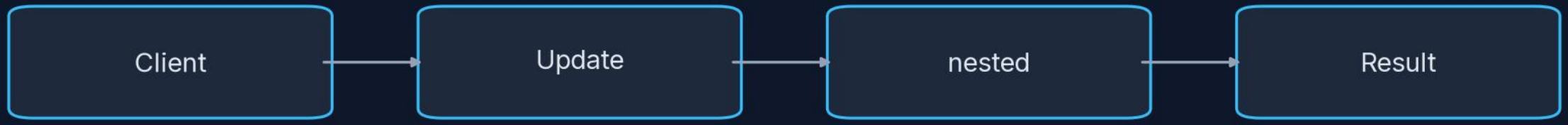
Today's Hook

- Today's force: Update nested city in immutable Person/Address
- We earn Lenses by feeling the naive pain first.
- Demo main prints a real JDK 25 trace (happy + failure).
- Key types: Address, Person, Lens, PersonLenses

The Problem

- Scenario: Update nested city in immutable Person/Address
- Constraints: solve «Update nested city in immutable Person/Address» with Lenses — keep the demo runnable and the failure path observable.
- Naive pain: Naive version fails to cleanly solve: Update nested city in immutable Person/Address
- Success looks like: {city:NYC}

Structure



Lenses: Update nested city in immutable Person/Address

Real-World Scenario

- Running example: Update nested city in immutable Person/Address
- Key types: Address, Person, Lens, PersonLenses
- Happy-path stdout cue: {city:NYC}
- Failure-path stdout cue: Failure path is asserted in ProblemTest (injected fault / invalid input)

Naive Code

```
package com.buildwithwaqar.patterns.functional.lens.problem;

import java.util.*;

public final class NaivePerson {
  public static String move(String personJson, String city){ return personJson.replace("Old", city); }
}
```

Why Naive Hurts

- Naive version fails to cleanly solve: Update nested city in immutable Person/Address
- Without Lenses, call sites absorb the wrong details.
- Failure mode to show on camera: Failure path is asserted in ProblemTest (injected fault / invalid input)
- New requirements force shotgun edits across the naive code.

Solution

```java
package com.buildwithwaqar.patterns.functional.lens.problem;

import java.util.*;
import java.util.function.*;

record Address(String city, String zip){}
record Person(String name, Address address){}
record Lens<S,A>(Function<S,A> get, BiFunction<S,A,S> set){
  A getA(S s){ return get.apply(s); }
  S setA(S s, A a){ return set.apply(s,a); }
  <B> Lens<S,B> andThen(Lens<A,B> other){
    return new Lens<>(s -> other.get.apply(get.apply(s)), (s,b) -> set.apply(s, other.set.apply(get.apply(s), b)));
  }
}
final class PersonLenses {
  static final Lens<Person,Address> address = new Lens<>(Person::address, (p,a)-> new Person(p.name(), a));
  static final Lens<Address,String> city = new Lens<>(Address::city, (a,c)-> new Address(c, a.zip()));
  static final Lens<Person,String> personCity = address.andThen(city);
}
```

Runnable main

```
package com.buildwithwaqar.patterns.functional.lens.problem;
import java.util.*; import java.util.function.*;
public final class ProblemDemo {
  public static void main(String[] args) {
    System.out.println("=== Lenses: nested city update ===");
    System.out.println("--- naive ---"); System.out.println(NaivePerson.move("{city:Old}", "NYC"));
System.out.println("--- lenses ---");
var p=new Person("Ada", new Address("Old", "10001"));
var moved=PersonLenses.personCity.setA(p, "NYC");
System.out.println(p); System.out.println(moved);
System.out.println("city="+PersonLenses.personCity.getA(moved));
  }
}
```

Walkthrough Output

```
=== Lenses: nested city update ===
--- naive ---
{city:NYC}
--- lenses ---
Person[name=Ada, address=Address[city=Old, zip=10001]]
Person[name=Ada, address=Address[city=NYC, zip=10001]]
city=NYC
```

Step-by-Step Trace

- Match stdout to stages — look for: {city:NYC}
- Then the injected failure: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Re-run: java -cp … 62-lenses-optics ProblemDemo (see episode folder).
- Pause the video and predict the next printed line.

Modern Java 25 / Zazr

```java
package com.buildwithwaqar.patterns.functional.lens.problem;

import java.util.*;
import java.util.function.*;

record Address(String city, String zip){}
record Person(String name, Address address){}
record Lens<S,A>(Function<S,A> get, BiFunction<S,A,S> set){
  A getA(S s){ return get.apply(s); }
  S setA(S s, A a){ return set.apply(s,a); }
  <B> Lens<S,B> andThen(Lens<A,B> other){
    return new Lens<>(s -> other.get.apply(get.apply(s)), (s,b) -> set.apply(s, other.set.apply(get.apply(s), b)));
  }
}
final class PersonLenses {
  static final Lens<Person,Address> address = new Lens<>(Person::address, (p,a)-> new Person(p.name(), a));
  static final Lens<Address,String> city = new Lens<>(Address::city, (a,c)-> new Address(c, a.zip()));
  static final Lens<Person,String> personCity = address.andThen(city);
}
```

Trade-offs

- Use Lenses when «Update nested city in immutable Person/Address» repeats and a simpler feature does not erase it.
- Refuse extra types when a sealed switch or a function value already suffices.
- Teaching demos omit production metrics, retries, and multi-region clocks.
- Interview soundbite: state the force behind Lenses in product language.

Related Patterns

- Name the pattern usually confused with Lenses and separate the forces.
- Sketch the roles for «Update nested city in immutable Person/Address» from memory in sixty seconds.
- Point to the next episode in this season on the playlist map.
- Link this episode's series/ folder in the YouTube description.

Recap

- Problem: Update nested city in immutable Person/Address
- Naive pain → Lenses boundary → Demo stdout.
- Remember the failure cue: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Break a ProblemTest assertion, watch it fail, restore it.